

Adopting Lightspeed's DeepAgents runtime in Konveyor

November 11, 2026

Runtime integration

- Konveyor with Goose
- The prototype with Lightspeed
- Changes in Lightspeed's runtime
- Coolstore planning demo
- Questions

Konveyor with Goose

1. **You choose an app and a task**

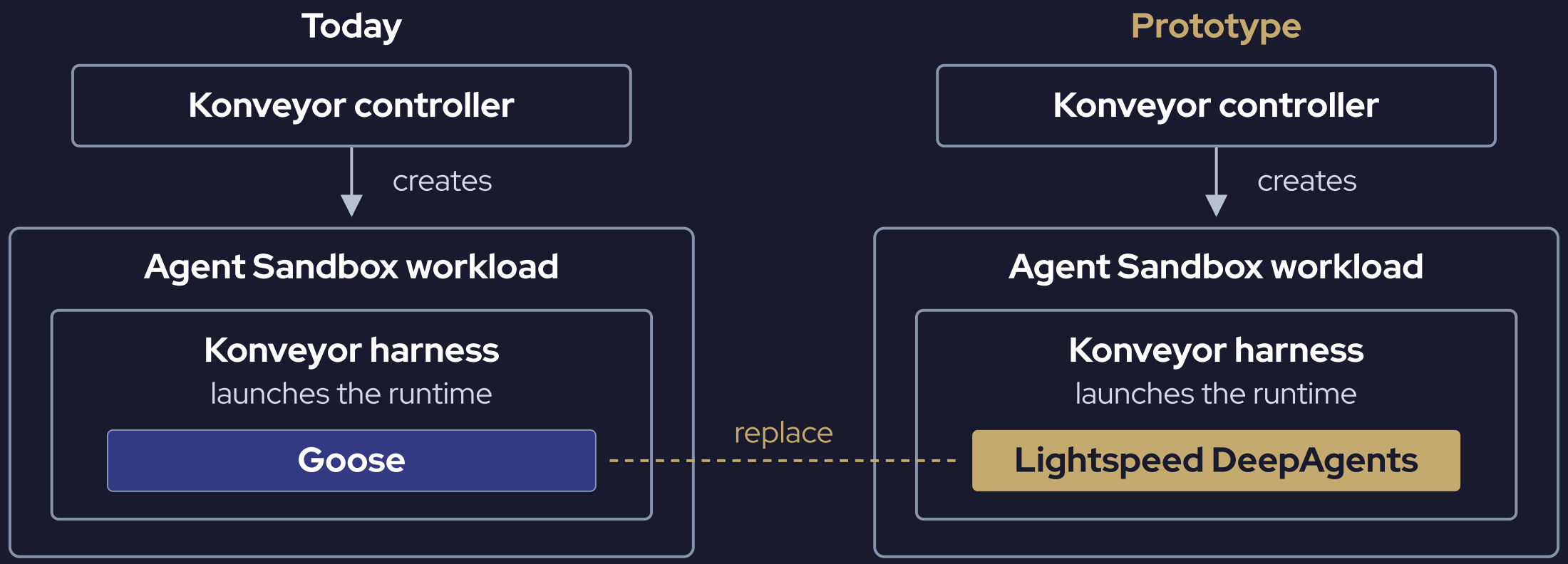
2. **Our harness prepares the repo**

3. **Goose works on the code**
   Calls the model, uses tools, commits locally

4. **Our harness verifies and pushes**
   You get the result on a Git branch



Konveyor with Lightspeed



Our harness still prepares the repo, verifies the work and pushes to Git.

Lightspeed runtime changes

- **Sessions:** send another prompt without starting over
- **Controls:** approve a tool, answer a question, give guidance and stop work
- **Optional ACP adapter:** connect sessions to our existing UI
- **Token counts:** send usage back to Konveyor
- **Model settings and errors:** pass reasoning effort through and report output cutoffs

We kept Lightspeed's checks on tool output.

Coolstore planning demo

A Java EE application we want to move to Quarkus

Play the separate demo video (2:44)

Runtime questions

- Would session support be useful to you?
- Does an optional ACP adapter make sense?
- How would you like to review these changes?

Code shown in the demo:

- Konveyor harness: djzager/agentic-controller
- Lightspeed runtime: djzager/lightspeed-agentic-sandbox
- Konveyor UI: djzager/tackle2-ui

We can pause here, or talk about your operator.

Lightspeed operator discussion

- Workload images and startup
- Placement and credentials
- Run completion and cancellation
- Questions and follow-up

Workload images and startup

We need to choose an image for each run.

- Different projects need different build tools
- Each image must start our harness
- The image's normal entrypoint may be enough

Today, the operator's image setting is shared across runs.

Placement and credentials

Hub and its Secrets live in our namespace.

Your operator uses its namespace for workloads and provider Secrets.

How would the workload get:

- Application data and credentials from Hub?
- Model settings and credentials?
- A connection from our UI for live interaction?

Run completion and cancellation

We need time to:

1. Stop the agent
2. Verify and push its work
3. Report what happened
4. Revoke the Hub token and clean up

What should users see if Git push succeeds but publishing the Result fails?

Operator questions

- Try two runs with different images
- Work through access to Hub and model credentials
- Test completion, cancellation and reporting failures
- Agree who wants to help and when to follow up

We can keep moving on the runtime while we work out operator integration.